LAB 8: FREQUENCY DOMAIN IDENTIFICATION OF QNET DC MOTOR

```
%--------------------------------------
% Fill in your student Name and ID.
Students.Name = 'Seyed Yazdan Zinati';
Students.ID = '260928077';
%--------------------------------------
```

0. Objectives

In this lab, we will use frequency analysis to identify the transfer function of the Virtual Motor. To initiate a connection with the virtual motor, run the following:

```
Motor = VirtualMotor();
```

1. Frequency Response Analysis

When a cosine wave $x(t) = M_x \cos(\omega t)$ is injected into a linear system, the system will respond with the same frequency ω, a certain magnitude M_x and a certain phase angle relative to the input ϕ. The steady-state system output can be written as $y(t) = M_y \cos(\omega t + \phi)$.

If $G(j\omega)$ is the frequency response of this system, we define the magnitude and phase at ω by:

$$M(\omega) = \frac{M_y}{M_x} = ||G(j\omega)||$$

$$\phi(\omega) = \angle G(j\omega)$$

1.1 Bode plot

The Bode plot presents the magnitude and the phase shift between the input and output at each frequency. It is possible to draw a `point-by-point` Bode plot by injecting a cosine wave with a known frequency, once at a time, and measuring the magnitude and phase shift of the output after it reaches its steady-state.

In a bode plot the magnitude is represented in a logarithmic scale in decibels (db).

$$M_{db}(\omega) = 20 \log(M(\omega))$$

The phase at each frequency can be found by measuring the temporal delay between the output and the input when the input is a cosine wave. Another way to measure the phase shift is through the Lissajous method. A Lissajous curve (an ellipsoid) is found when we try to plot the output in function of the input (XY plot). In fact, we can show that at a frequency ω

$$\cos(\phi) = \frac{y^*}{M_y}$$

Where y^* is the value of the output y when the input x is at the maximum M_x.

An illustration of using the Lissajous method (taken from Wikipedia) is shown below:

<<fig_lab08_lissajousphase.png>>

Question 1 (2.5 mark)

- Define the following transfer function in **MATLAB**

$$G(s) = \frac{s+4}{s^2+s+1}$$

```
s = tf('s');
G = (s + 4)/(s^2 + s + 1)
```

```
G =

     s + 4
  -----------
  s^2 + s + 1

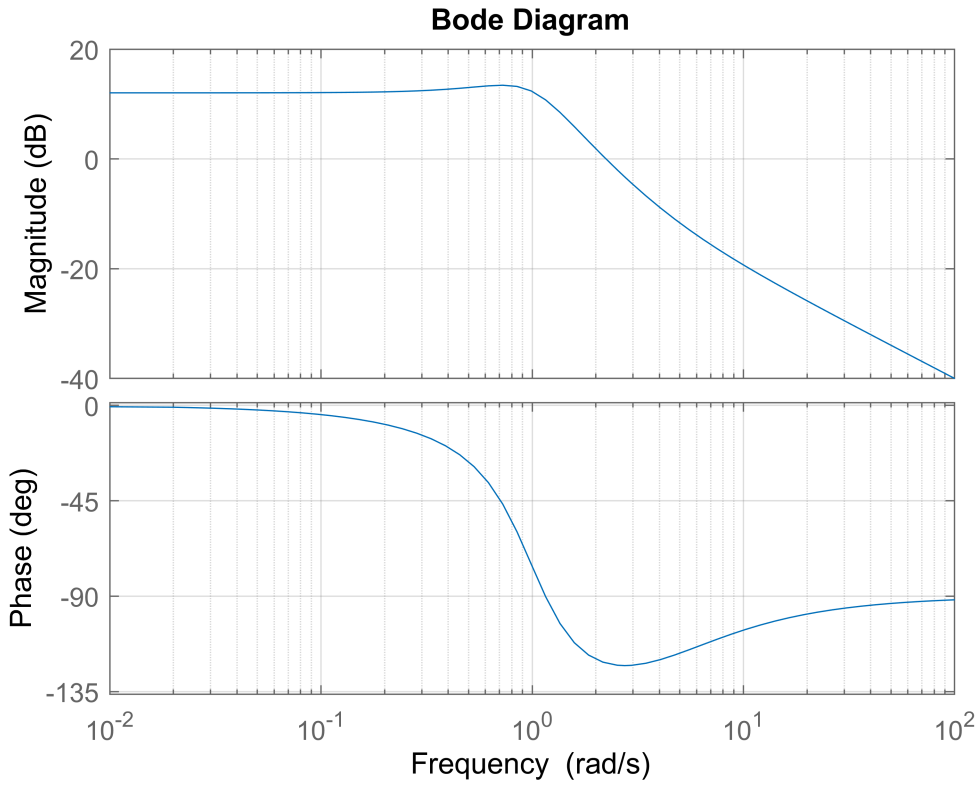
Continuous-time transfer function.
```

- Plot the bode graph of this system. (Hint: help **bode**)

```
figure(1); clf;

bode(G)

grid on
```



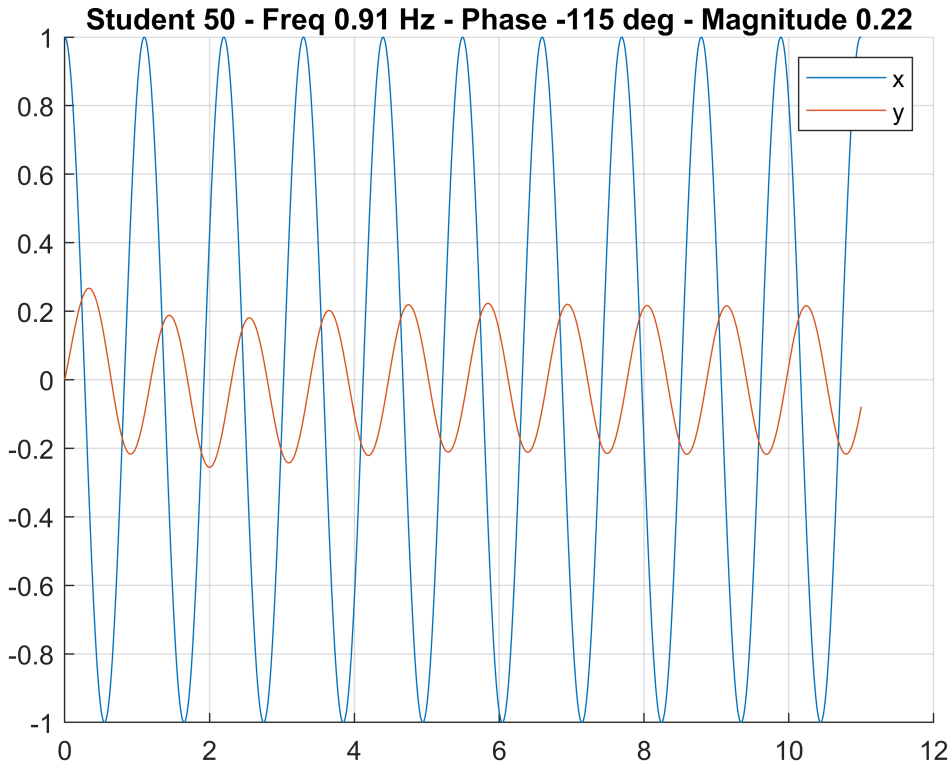
- Consider the following signal:

```
rng(datetime('now').Year*randi([1,20],1))
Freq = round(0.1+rand(1), 2);
t = 0:0.01:round(10/Freq);
x = cos(2*pi*Freq*t);
```

Using **`lsim`** simulate the output $y(t)$ representing the response of the system G to the signal $x(t)$.

```
y = lsim(G, x, t);

figure(2); clf;
title(sprintf('Student %d - Freq %3.2f Hz - Phase %+03d deg - Magnitude %4.2f',...
    Students.ID(1), Freq, round(180*angle(evalfr(G, 2*pi*Freq*1i))/pi), abs(evalfr(G, 2*pi*Freq
hold on
plot(t, x)
plot(t, y)
grid on
legend('x', 'y')
```

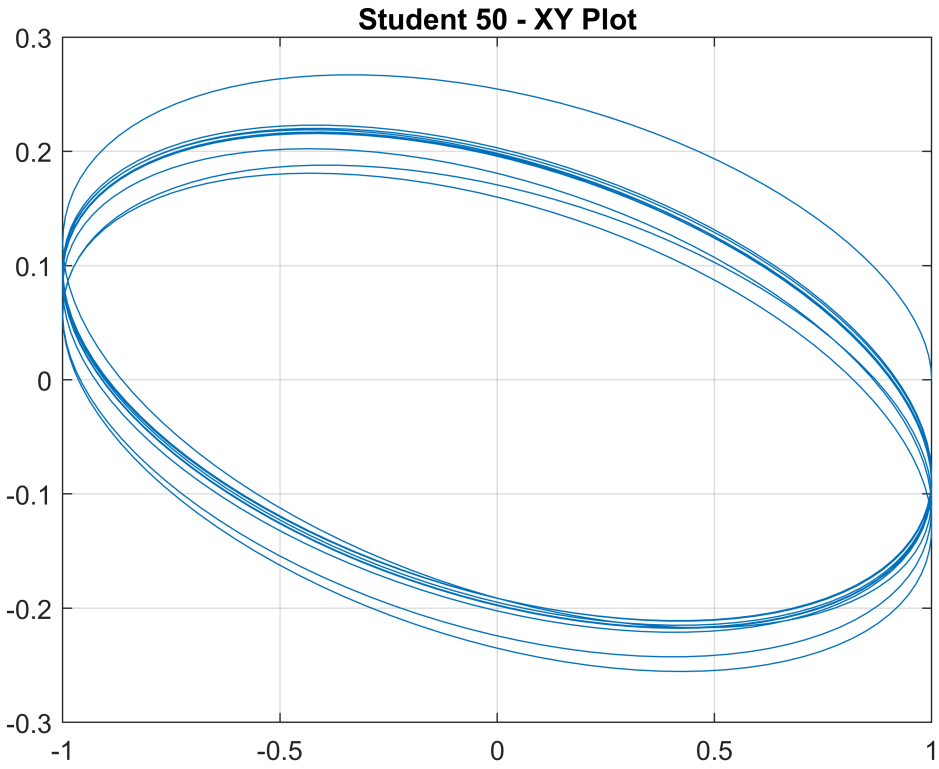


- Plot $y(t)$ in function of $x(t)$.

```
figure(3); clf;

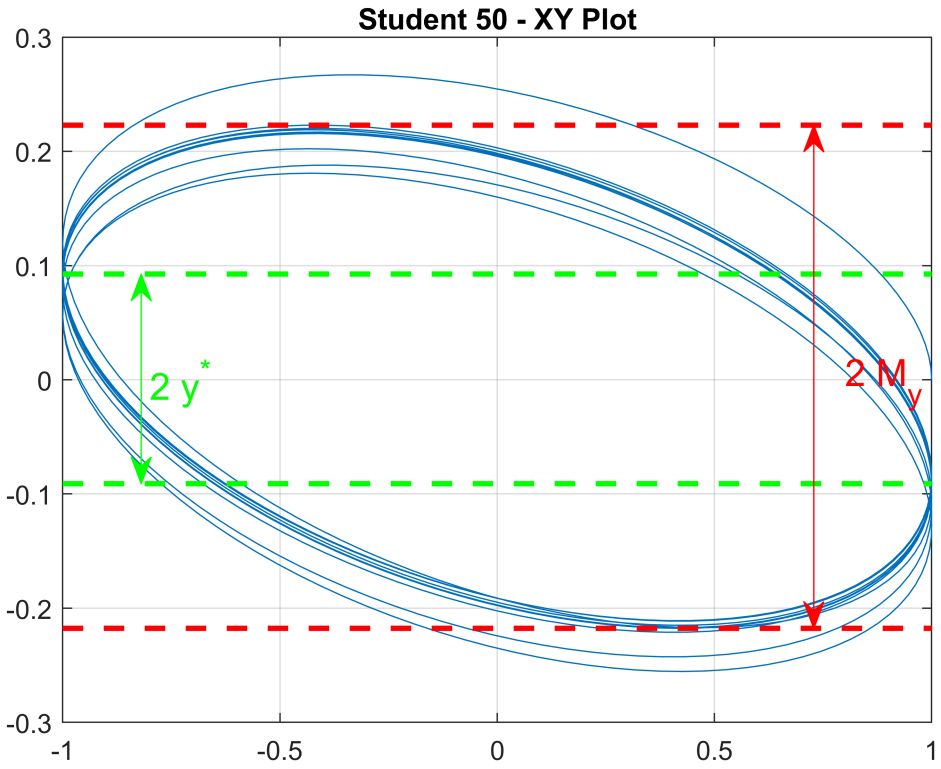
plot(x, y)

title(sprintf('Student %d - XY Plot', Students.ID(1)));
grid on
```



- Find graphically the phase ϕ and magnitude M at the specified frequency.

```
showMyAndYStar(t, x, y, Freq);
```




```
y_star = (0.07003 - (-0.07043)) / 2;
M_y = (0.1883 - (-0.1845)) / 2;

% phase
cos_phi = y_star / M_y;
phi = -180 + acosd(cos_phi)
```

```
phi = -112.1338
```

```
% magnitude
M = 2*M_y / (0.451185 + 0.434345)
```

```
M = 0.4210
```

3. Identification of Qnet DC Motor Frequency Response

To avoid the non-linear domain of the DC Motor at low voltage we will stimulate the DC Motor with offsetted cosine waves. We choose the following voltage signal:

$$v(t) = 4 + 2\cos(2\pi ft)$$

To send a cosine wave of frequency $f = 0.5$ Hz to the Virtual Motor we do the following:

```
dt = 0.01; % sampling time
T = 15; % simulation duration
t = 0:dt:T; % define the time signal
vAmp = 2; % cosine amplitude
```

```
vOffset = 4; % voltage offset
Freq = 0.5; % frequency of cosine wave
Motor.reset();
Motor.input(vOffset + vAmp * cos(2*pi*Freq*t));
Motor.off();
Motor.save(Students, 'Lab08.Sinewave');
Motor.load('Lab08.Sinewave');
```

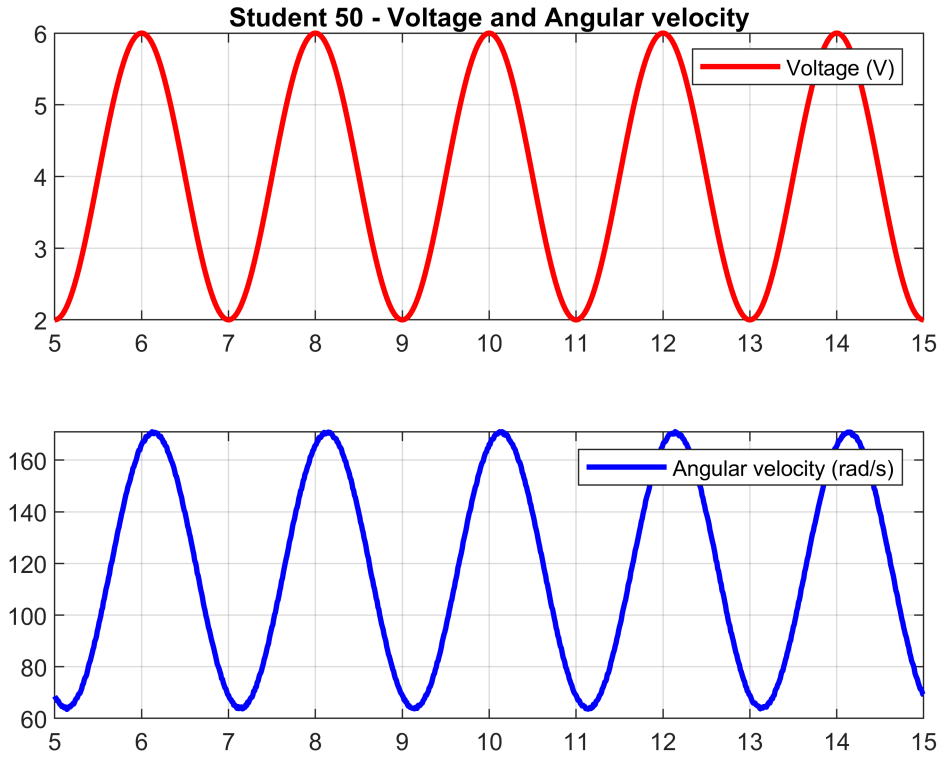
```
VirtualMotor::load::info: Recovering data collected by students: [260928077] .
```

For frequency analysis we are only interested at the steady-state response. We can retrieve the resulting angular velocity at steady state by:

```
delay = 5; % Ignore the first 5 seconds of experiment (transient response)
w = Motor.velocity(t >= delay);
v = Motor.voltage(t >= delay);
```

The following shows a plot of the temporal response.

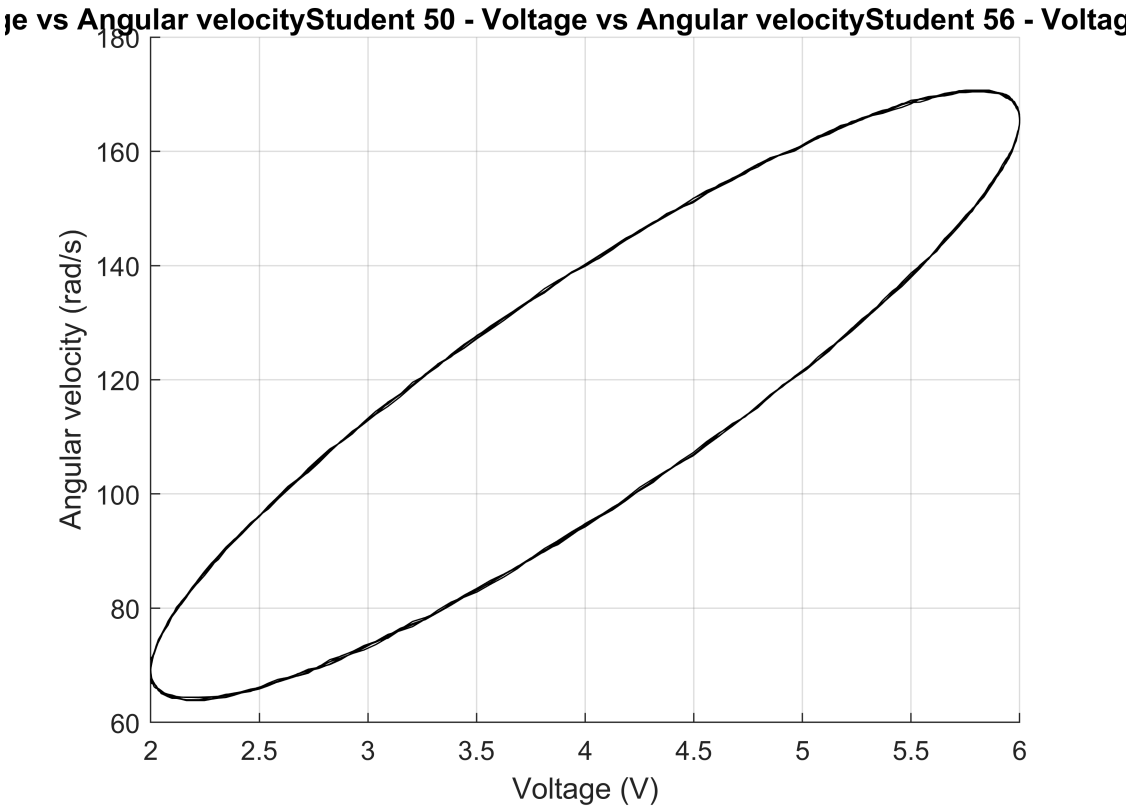
```
figure(4), clf;
subplot(2, 1, 1);
plot(t(t >= delay), v, 'r', 'linewidth', 2.0);
legend('Voltage (V)')
grid on
xlim([delay, T])
title(sprintf('Student %d - Voltage and Angular velocity', Students.ID(1)));
subplot(2, 1, 2);
plot(t(t >= delay), w, 'b', 'linewidth', 2.0);
legend('Angular velocity (rad/s)')
grid on
xlim([delay, T])
```



And the following shows the XY plot, which we can use to find y^*, M_y and M_x.

```
w_filter = medfilt1(w, 10, 'truncate'); % we can filter the noise if needed

figure(5), clf; hold on;
title(sprintf('Student %d - Voltage vs Angular velocity', Students.ID));
plot(v, w_filter, 'k');
grid on
xlabel('Voltage (V)')
ylabel('Angular velocity (rad/s)')
xlim([2 6])
```



Question 3 (3.0 marks)

Find the magnitude and phase of the transfer function of the Virtual Motor for the following frequencies.

```
FreqVect = [0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 2.0, 4.0, 6.0];

dt = 0.01;  % sampling time
T = 15; % simulation duration
t = 0:dt:T;  % define the time signal
vAmp = 2  % cosine amplitude
```

```
vAmp = 2
```

```
vOffset = 4  % voltage offset
```

```
vOffset = 4
```

```
Freq = 6  % frequency of cosine wave (a frequency from FreqVect)
```

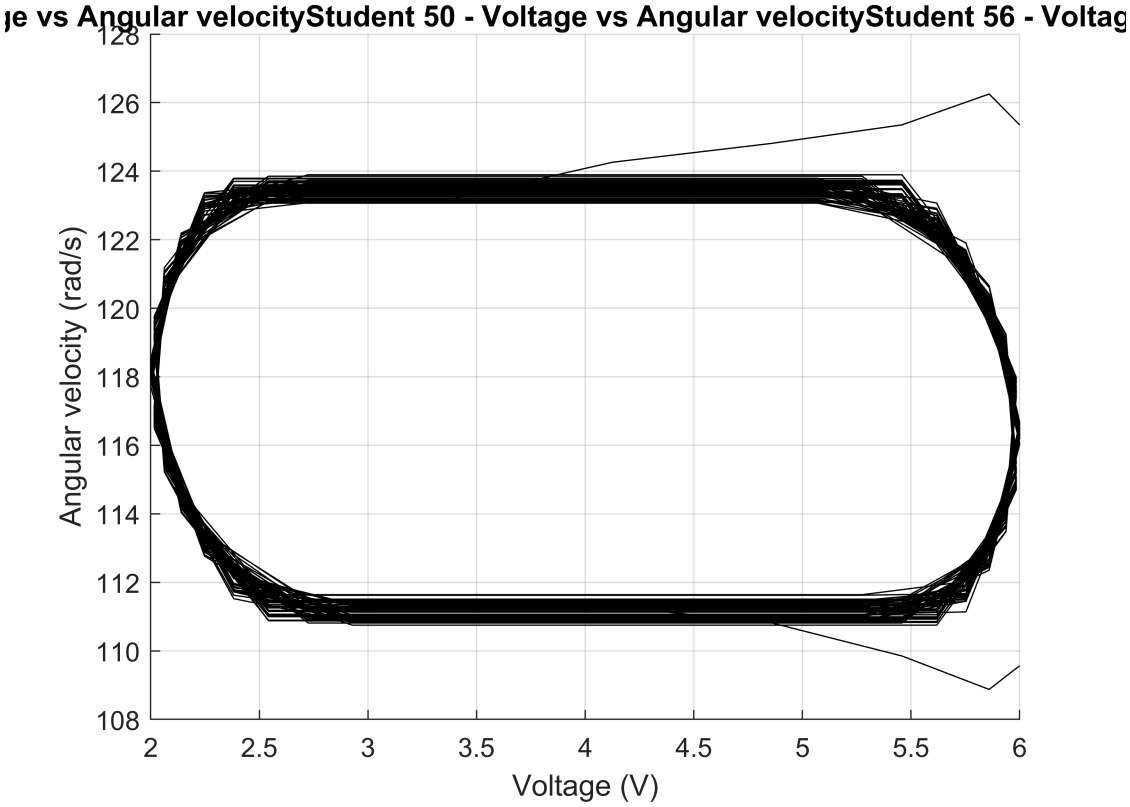
```
Freq = 6
```

```
Motor.reset();
Motor.input(vOffset + vAmp * cos(2*pi*Freq*t));
Motor.off();
Motor.save(Students, 'Lab08.Sinewave');
Motor.load('Lab08.Sinewave');
```

```
VirtualMotor::load::info: Recovering data collected by students: [260928077] .
```

```
delay = 5; % Ignore the first 5 seconds of experiment (transient response)
w = Motor.velocity(t >= delay);
v = Motor.voltage(t >= delay);
w_filter = medfilt1(w, 10, 'truncate'); % we can filter the noise if needed

figure(6), clf; hold on;
title(sprintf('Student %d - Voltage vs Angular velocity', Students.ID));
plot(v, w_filter, 'k');
grid on
xlabel('Voltage (V)')
ylabel('Angular velocity (rad/s)')
xlim([2 6])
```



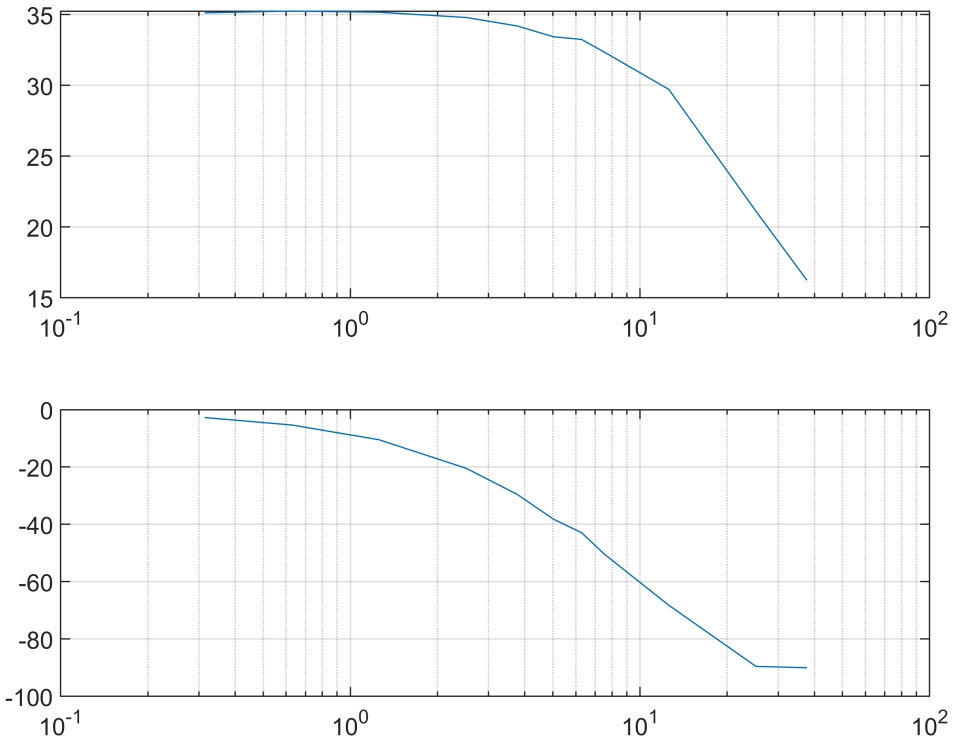
Measure graphically the phase and magnitude for each frequency and put it here. Note: The size of MagnitudeDB and PhaseDeg is the same as FreqVect.

```
% Answer here
% 20*log10(M) we get:
MagnitudeDB = [35.12, 35.24, 35.16, 34.79, 34.19, 33.43, 33.24, 32.34, 29.72, 21.12, 16.24];
PhaseDeg = - [2.75, 5.35, 10.5, 20.46, 29.56, 38.2, 42.9, 50.43, 68.2, 89.53, 90 ];
```

Question 4 (1.0 mark)

Draw the resulting Bode plot (Magnitude (dB) vs Frequency (Hz)) on a semilog plot and Phase (deg) vs Frequency (Hz) on a semilog plot). (Hint: help **`semilogx`**)

```
omega = 2 * pi * FreqVect;
figure(7); clf;
subplot(2, 1, 1)
% Draw magnitude here
semilogx(omega,MagnitudeDB)
grid on
subplot(2, 1, 2)
% Draw Phase here
semilogx(omega,PhaseDeg)
grid on
```



Question 5 (1.0 mark)

From the Bode plot, measure the DC gain.

```
DC_gain = 10^(35.12/20)
```

```
DC_gain = 57.0164
```

Question 6 (1.0 mark)

From the Bode plot, measure the cut-off frequency (Frequency at which the magnitude drops by a factor of $-20\log_{10}\left(\frac{1}{\sqrt{2}}\right) = -3$ (dB) from the DC gain).

```
w_cutoff = 8.09342
```

```
w_cutoff = 8.0934
```

Question 7 (1.5 mark)

For a first order system the transfer function is expressed by

$$H(s) = \frac{DC_{gain}}{1 + \frac{s}{\omega_{cutoff}}}$$

- Write the transfer function of the identified DC Motor.

```
H =  DC_gain / (1 + (s/w_cutoff))
```

```
H =

    461.5
  ---------
  s + 8.093

Continuous-time transfer function.
```

- Complete the code to superpose the experimental bode plot with the bode plot of the identified transfer function.

```
H_Freq = squeeze(freqresp(H, omega, 'rad/s'))';

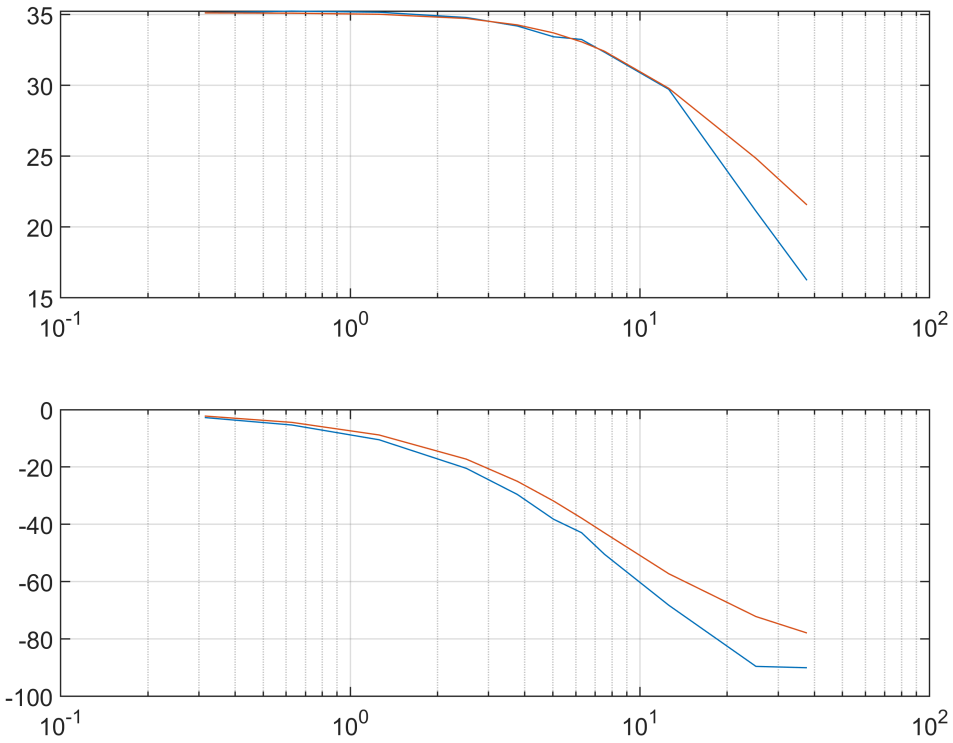

H_MagnitudeDB =  20*log10(abs(H_Freq)) % find magnitude in db
```

```
H_MagnitudeDB = 1×11
   35.1135   35.0939   35.0165   34.7202   34.2672   33.7032   33.0715   32.4065 ···
```

```
H_PhaseDeg =  -rad2deg(angle(H_Freq)) % find phase in degrees
```

```
H_PhaseDeg = 1×11
   -2.2229   -4.4392   -8.8256  -17.2513  -24.9761  -31.8430  -37.8234  -42.9719 ···
```

```
figure(9)
subplot(211)
semilogx(omega, [MagnitudeDB; H_MagnitudeDB])
grid on
subplot(212)
semilogx(omega, [PhaseDeg; H_PhaseDeg])
grid on
```



- How accurate is the point-by-point bode plot for high frequencies? Comment on your results.

```
% The point-by-point bode plot is quite accurate when working
% with small frequencies, it becomes less accurate for higher frequencies.
% This can bee seen as the phase and magnitude don't exactly superpose.
% This is due to the fact that the accuracy is inversly proportional to the
% thickness of the curve. Therefore: as we increase the frequency, the thickness of
% the curve increases, and our accuracy drops.
```

4. Useful Functions

4.1 Customize the XY plot

```
function showMyAndYStar(t, x, y, Freq)
max_y = max(y(t > round(5/Freq)));
max_x = max(x(t > round(5/Freq)));
min_y = min(y(t > round(5/Freq)));
min_x = min(x(t > round(5/Freq)));
star_y_1 = mean(y(t > round(5/Freq) & abs(x-max_x) < 0.05));
star_y_2 = mean(y(t > round(5/Freq) & abs(x-min_x) < 0.05));

gcf; hold on; axPos = get(gca, 'Position'); xMinMax = xlim; yMinMax = ylim;
plot([min_x, max_x], [max_y, max_y], '--r', 'linewidth', 2.0)
plot([min_x, max_x], [min_y, min_y], '--r', 'linewidth', 2.0)
annotation('doublearrow', [0.8, 0.8],...
```

```
    axPos(2)+(([max_y, min_y] - yMinMax(1)) / (yMinMax(2) - yMinMax(1)))*axPos(4), 'Color', 're
text(0.8, 0.0, '2 M_y', 'Color', 'red', 'FontSize', 14)
plot([min_x, max_x], [star_y_1, star_y_1], '--g', 'linewidth', 2.0)
plot([min_x, max_x], [star_y_2, star_y_2], '--g', 'linewidth', 2.0)
annotation('doublearrow', [0.2, 0.2],...
    axPos(2)+(([star_y_1, star_y_2] - yMinMax(1)) / (yMinMax(2) - yMinMax(1)))*axPos(4), 'Color
text(-0.8, 0.0, '2 y^*', 'Color', 'green', 'FontSize', 14)
end
```